TUTORIAL – 9

Black-Box & White-Box Testing,

Integration Testing, and System Testing on a Sample Web Application

Aim: Analyze Black-Box Testing, White-Box Testing, Integration Testing, and System Testing on a Sample Web Application.

Sample Web Application: Meesho (https://www.meesho.com)

Tester Name: Roshni Rajput

Date: 18-09-2026

Webpage URL: https://www.meesho.com

Problem Statement

Perform Black-Box Testing, White-Box Testing, Integration Testing and System Testing on the Meesho e-commerce web application. Capture UI screenshots for functional flows and browser console logs for white-box analysis. Document test cases, results, defects and recommendations.

1. Black-Box Testing

Black-Box Testing examines the functionality of an application without peering into its internal structures or workings. Test cases are derived from requirements and specifications. On Meesho we validated homepage load, product search, category navigation/filters, product detail page, price/discount display, Add-to-Cart / Buy-Now actions and available offer information.

2. White-Box Testing

White-Box Testing inspects internal logic, code paths and browser-side behaviour. Using Chrome DevTools Console, the tester records CORS policy violations, failed static resource loads, HTTP/API errors, JavaScript warnings, service-worker activity and other client-side messages observed during the Meesho test session.

3. Integration Testing

Integration Testing verifies that modules work together. We tested the flow Search → Category/Filter → Product Detail Page and verified that the selected product, displayed price and available purchase/offer information remain consistent across the flow.

4. System Testing

System Testing evaluates the complete application against requirements – UI consistency of header/navigation, product grid, banners, category navigation, Login/Cart controls and the overall shopping journey.

Black-Box Test Cases & Screenshots

Scenario ID: BB-1a

Scenario: Homepage loads successfully

Test Steps:

1. Open https://www.meesho.com
2. Observe page load and major UI sections

Expected: Homepage loads with header/navigation, search bar, category/navigation elements and promotional/product sections.

Actual: Homepage loaded and the main shopping sections were available during the prepared test flow.
Status: PASS

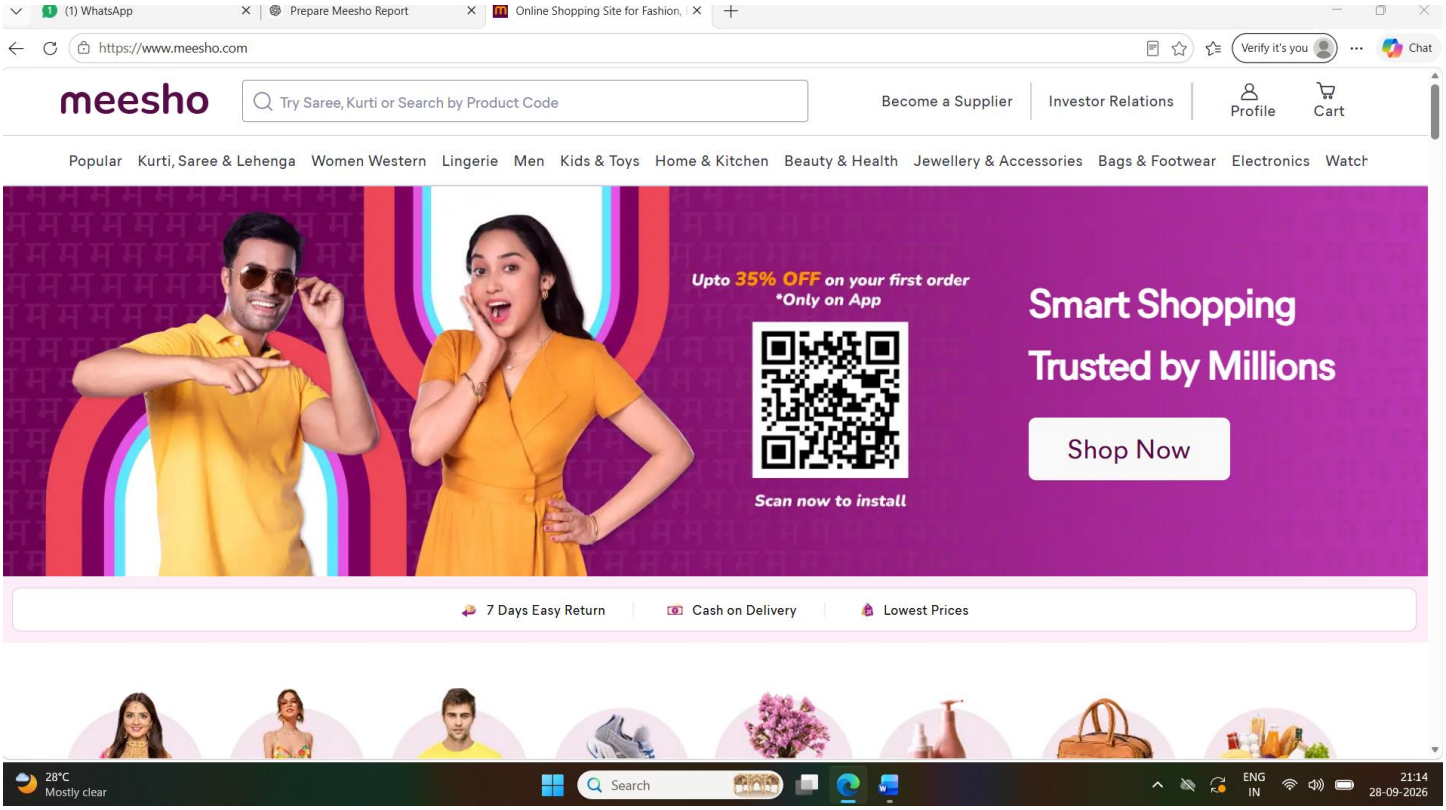


Figure 1 – Meesho Homepage (Screenshot to insert: https://www.meesho.com)

Scenario ID: BB-1b / BB-1c

Scenario: Search for “women kurti” & apply category/filter options

Test Steps:

1. Click the search bar and type “women kurti”
2. Press Enter
3. Use the available category/filter controls to narrow the product results

Expected: Search results show relevant products and the available category/filter controls work correctly.

Actual: Relevant product results and filtering/navigation controls were available in the prepared flow. Status: PASS

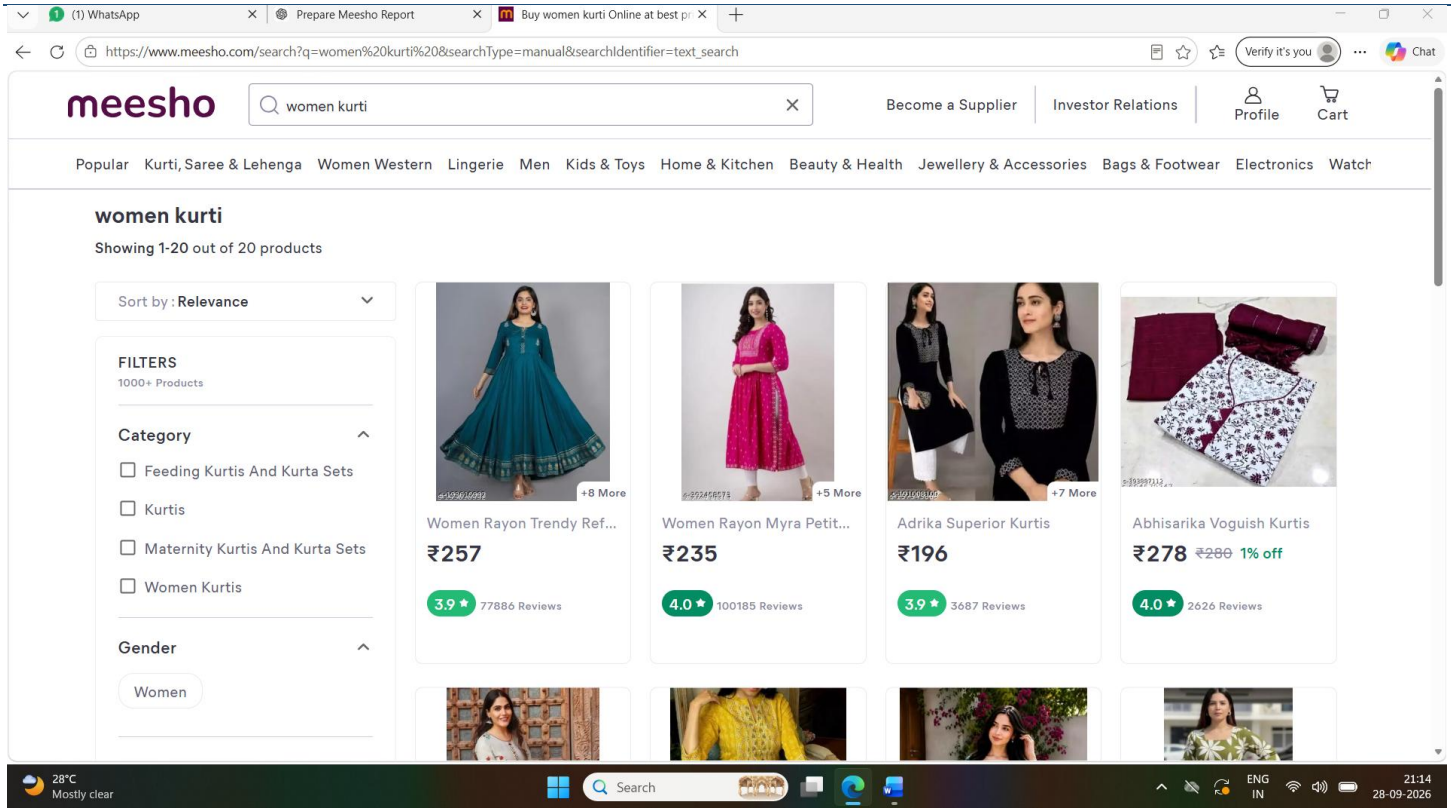


Figure 2 – Meesho Search Results (Search performed: “women kurti”; screenshot to insert)

Scenario ID: BB-1e / BB-1f / BB-1g

Scenario: Product Detail Page – price, offers, action buttons

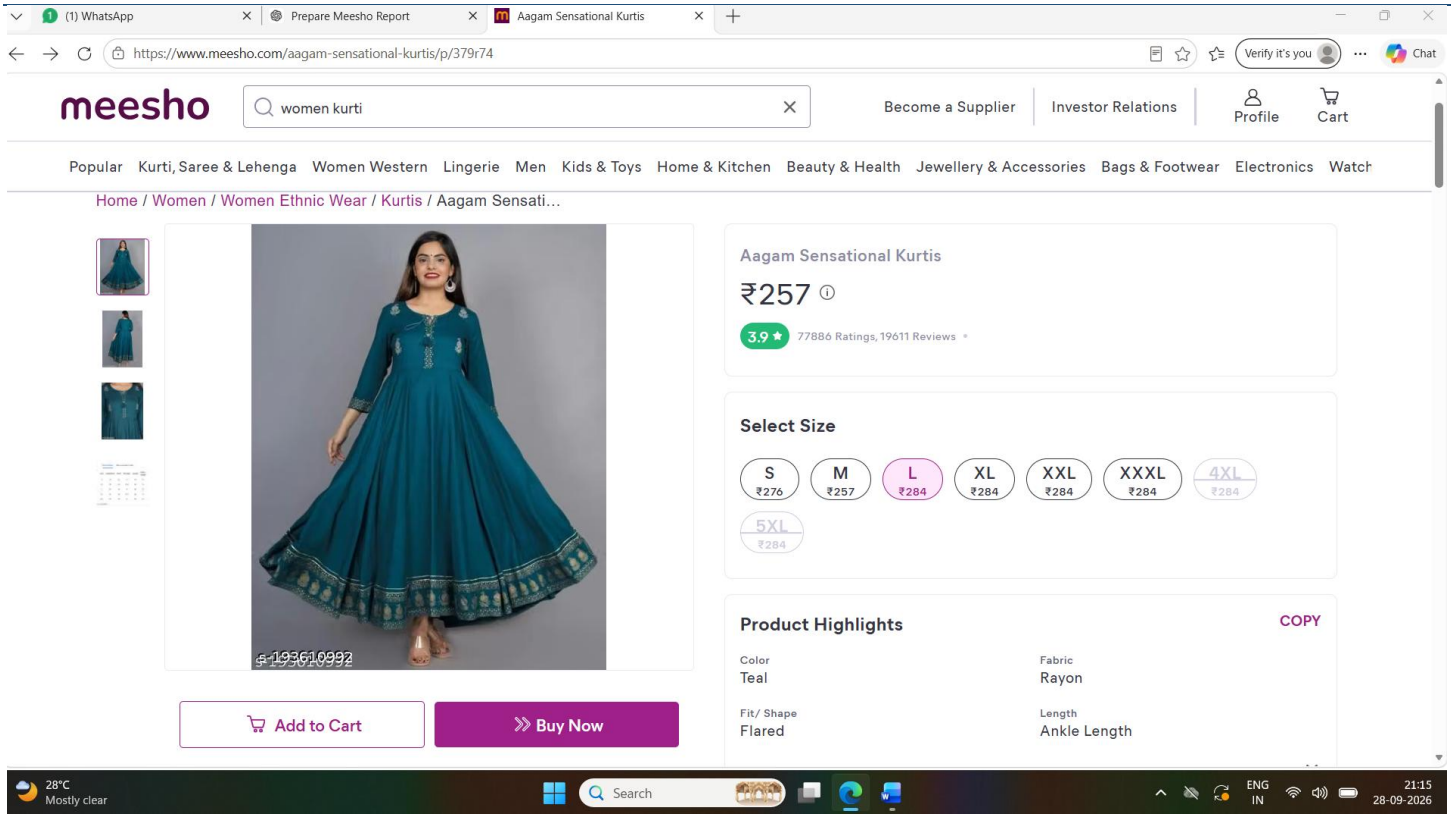
Test Steps:

1. Click any product (Women Printed Kurti)
2. Observe images, price, discount %, bank offers, Add to Cart & Buy Now

Scenario: Product Detail Page – price, offers and action buttons

Actual: All elements present and correct. Status: PASS

1. Click any product from the search results
2. Observe product images, price, discount information, delivery/offer information and Add to Cart / Buy Now controls



Expected: Product information is displayed consistently and the available action buttons are functional.

Scenario ID: WB-2a

Actual: Product information and purchase controls were available in the prepared flow. Status: PASS

Test Steps:

Figure 3 – Meesho Product Detail Page (price, offers and purchase actions)

2. Open Chrome DevTools → Console tab
3. Record all red/yellow messages

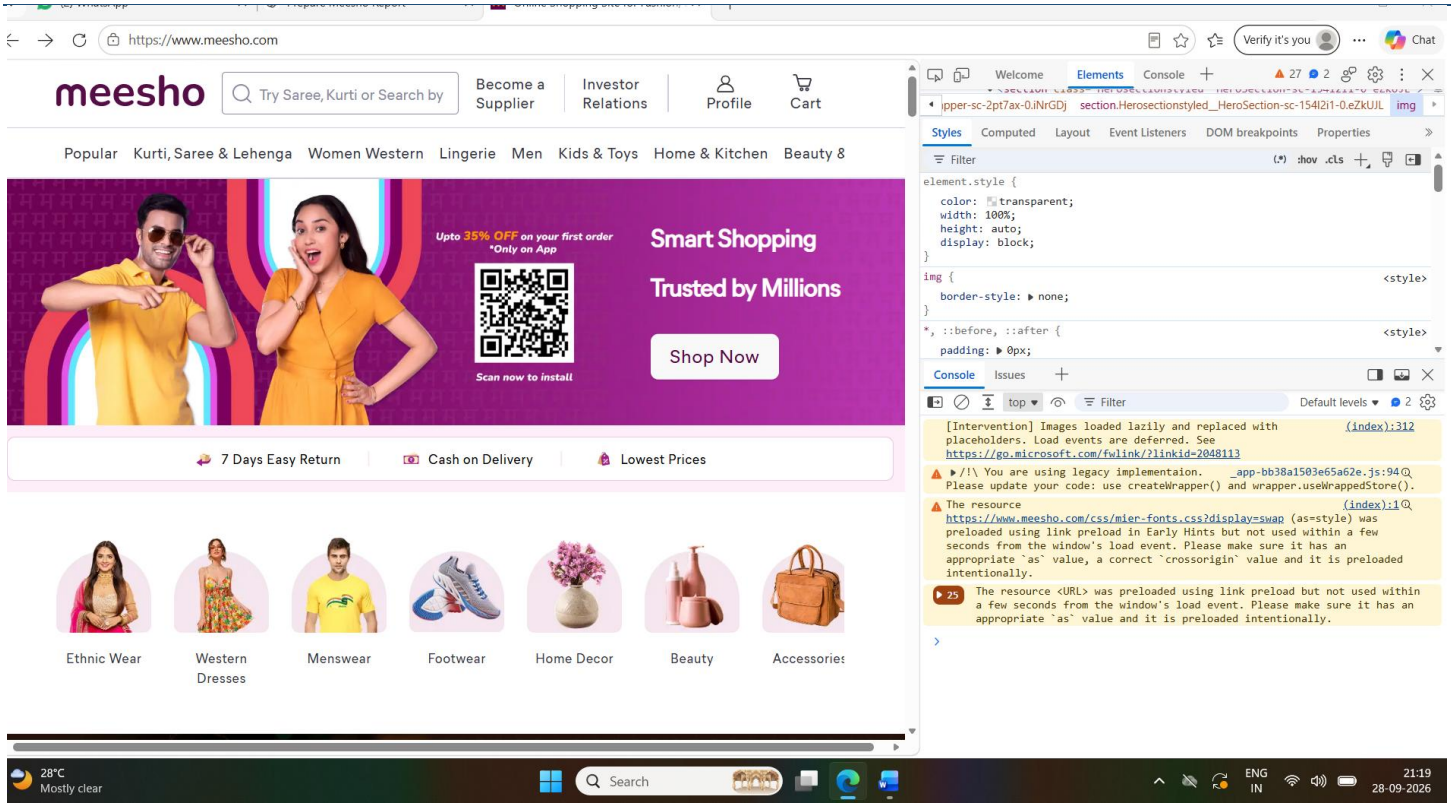
Expected: No critical CORS or resource-load failures.

Actual: Multiple CORS blocks on L1Nav/*.svg assets (x-user-agent header not allowed) and net::ERR_FAILED. Status: FAIL

Expected: No critical CORS, JavaScript or resource-load failures.

Actual: Record the Meesho Console output from the actual test session here. Status: TO BE VERIFIED

Figure 4 – Meesho Console errors/warnings on Homepage (Insert screenshot)



Test Steps:

1. Navigate to women kurti listing
2. Open Console
3. Observe network-related errors

Scenario ID: WB-2a

Scenario: Console analysis on Homepage load

1. Open Meesho homepage

2. Open Chrome DevTools → Console tab

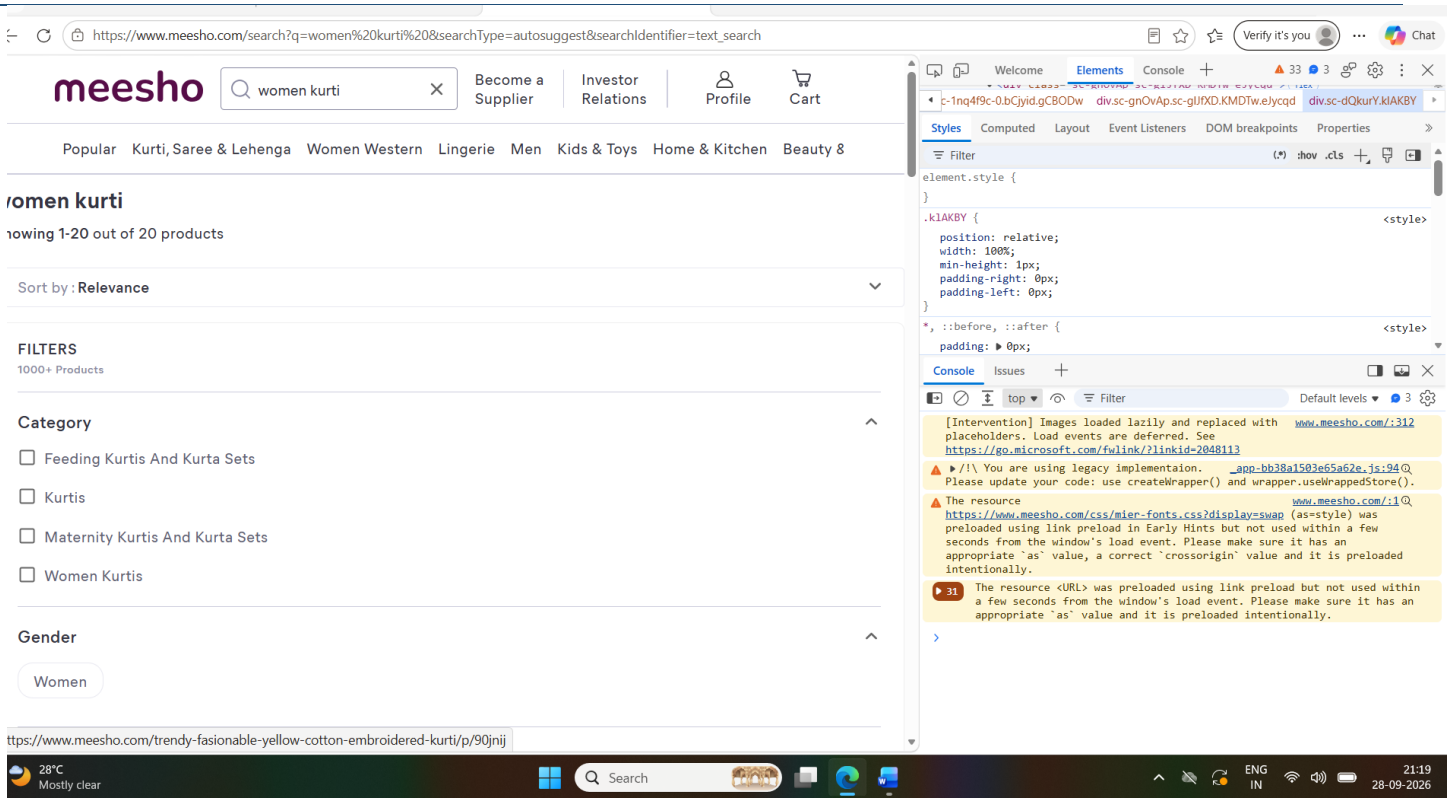
3. Record all red/yellow messages

Expected: No critical CORS, JavaScript or resource-load failures.

Actual: Console output must be captured from the actual Meesho test session. Status: TO BE VERIFIED

2. Open Console

Figure 4 – Meesho Console on Homepage (Insert screenshot)



Expected: Minimal warnings; no blocking errors.

Actual: Deprecated meta tag, missing RN components (Placeholder, RNShimmeringView, GifView…), CORS on codes.json, 406 on collector API. Status: FAIL

Scenario ID: WB-2b

Scenario: Console analysis on Search / Listing page

IT-3a Search → Category/Filter → PDP integration – PASS (selected product correctly linked from the filtered/listing results).

1. Search for “women kurti” and open the listing page
2. Open Console
3. Observe network-related errors and warnings

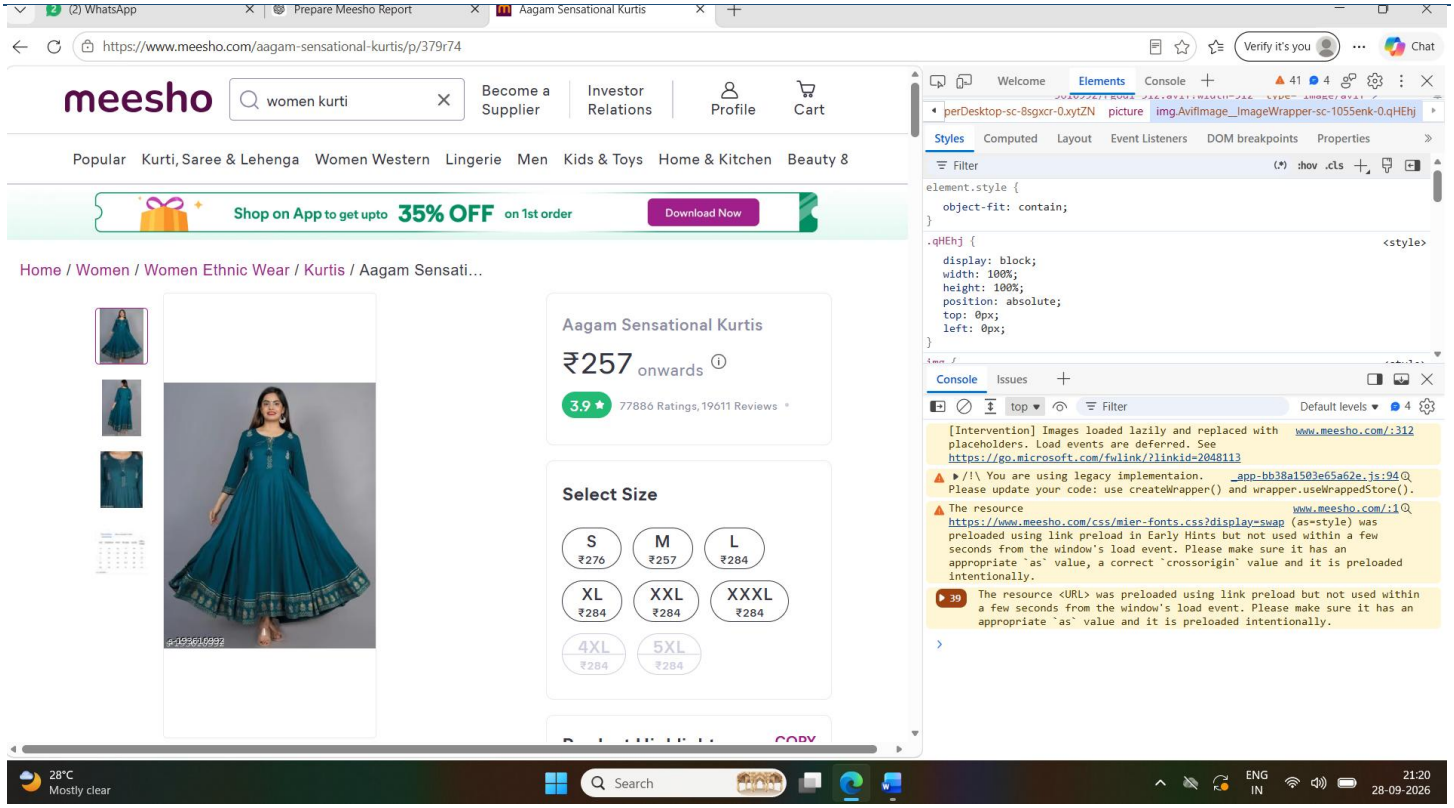
Expected: Clean console or only non-critical warnings.

Total Test Cases	Passed	Failed	Pass %
5	3	2	60.00%

Actual: Record any Meesho API/network errors from the actual session. Status: TO BE VERIFIED

1. Browser console CORS/resource errors (High) – verify and document only if reproduced during the Meesho test session.

Figure 5 – Meesho Console on Listing page (Insert screenshot)



Scenario ID: WB-2c
Scenario: Console analysis on Product Detail page

3. Internal Server Error (Medium) – verify and document the exact endpoint and status code if reproduced.
1. Open any Meesho product detail page

2. Open Console
3. Note warnings and errors

Expected: Minimal warnings; no blocking errors.

Bug ID	BUG-WB-1
Title	Legacy implementation warning in Meesho web application
Module	Homepage / Search / Product Detail
Severity	Low
Steps to Reproduce	1. Open https://www.meesho.com 2. Open DevTools → Console 3. Observe the legacy implementation warning
Expected Result	The application should use the current implementation without legacy implementation warnings.
Actual Result	Console displays a legacy implementation warning: **“You are using legacy implementation. Please update your code: use createWrapper() and wrapper.useWrappedStore().”**

Actual: Record product-page console output from the actual session. Status: TO BE VERIFIED

Bug ID	BUG-WB-2
Title	Preloaded font resource not used shortly after page load
Module	Homepage / Search / Product Detail – Resource Loading

Severity	Low
Steps to Reproduce	1. Open https://www.meesho.com 2. Open DevTools → Console 3. Observe the resource-loading warning for mier-fonts.css
Expected Result	Preloaded resources should be used shortly after page load or should not be unnecessarily preloaded.
Actual Result	Console displays a warning that **mier-fonts.css?display=swap was preloaded using link preload but was not used within a few seconds from the window's load event.**

Recommendations:

1. Review any CORS configuration reported in the browser console and ensure required request headers are permitted.
2. Investigate any failed analytics/API requests recorded in the console and provide graceful error handling.
3. Resolve any missing client-side component registrations or remove unused references that generate warnings.
4. Review browser deprecation warnings and replace deprecated meta tags or APIs where applicable.

— End of Report —